

Hey, I have a Lab 4 VibeCoding assignment. I have a Pygame vertical platformer called Climber. I need to fix one existing bug and implement three optional features from the README. I'll upload my game.py. Please first understand the existing code and explain what the game currently does, where the bug is, and what the three empty functions are supposed to do. Please don't change the code yet. I want to work through the tasks one by one because the lab requires separate commits for each task.

Absolutely, buddy. Upload your `game.py` (and ideally the README too, if it's separate).

I'll **only analyze it first** — **no code changes yet**. I'll break it down into:

1. 🎮 **What the current Climber game does**
 - Player movement
 - Jumping/gravity
 - Platforms
 - Coins/score
 - Camera/level progression
 - Game-over/win behavior
2. 🦎 **The existing bug**
 - Where it occurs in the code
 - Why it happens
 - What the expected behavior should be
 - What we should change later for the bug-fix commit
3. 🧩 **The three empty functions**
 - What each function is intended to implement according to the README
 - What inputs/outputs they have
 - Where each one connects to the rest of the game
 - A simple implementation plan for each
4. 📌 **Lab 4 task/commit order**
 - Task 1: bug fix
 - Task 2: optional feature
 - Task 3: optional feature
 - Task 4: optional feature
 - What should be committed separately

I won't modify the code in this step. Once you understand the existing code and agree with the first task, we'll implement **one task at a time** so your Git history matches the lab requirements.

Upload `game.py` whenever you're ready.

```
import random
import pygame
```

```
WIDTH, HEIGHT = 480, 640
FPS = 60
BG = (20, 20, 40)
GRAVITY, JUMP_SPEED, MOVE_SPEED = 0.5, -13, 4
PLAYER_W, PLAYER_H = 36, 36
PLATFORM_H = 14
COIN_R = 7
LIVES_START = 3
```

```
def platform_color(index, total):
    """Return an (r, g, b) colour override for the platform at this
    index (0 is the ground), or None for the default green."""
    pass
```

```
def moving_platform_speed(index, total):
    """Return a horizontal oscillation speed in pixels/frame for the
    platform at this index, or None/0 to keep it static."""
    pass
```

```
def on_coin_collected(coin, score):
    """Called the instant the player collects a coin, after its value has
    been added to the score. Add a sound or sparkle here."""
    pass
```

```
class Platform:
    def __init__(self, index, total, x, y, w=120, movable=True):
        self.rect = pygame.Rect(x, y, w, PLATFORM_H)
        self.speed = (moving_platform_speed(index, total) or 0) if
movable else 0
        self.bounds = (max(0, x - 60), min(WIDTH - w, x + 60))
        self.color = platform_color(index, total) or (100, 180, 100)
```

```
def update(self):
    if not self.speed:
        return 0
    dx = self.speed
    if self.rect.x <= self.bounds[0] or self.rect.x >= self.bounds[1]:
        self.speed = -self.speed
        dx = self.speed
    self.rect.x = max(self.bounds[0], min(self.bounds[1], self.rect.x
+ dx))
    return dx

def draw(self, screen, cam_y):
    pygame.draw.rect(screen, self.color, self.rect.move(0, -cam_y),
border_radius=4)

class Coin:
    def __init__(self, x, y):
        self.pos = pygame.Vector2(x, y)
        self.taken = False

    def draw(self, screen, cam_y):
        pygame.draw.circle(screen, (250, 210, 60), (self.pos.x, self.pos.y
- cam_y), COIN_R)

def generate_platforms(num, start_y, width):
    platforms = [Platform(0, num, 0, start_y, width, movable=False)]
    # ground never moves
    y = start_y - 100
    for i in range(1, num + 1):
        x = random.randint(20, width - 140)
        w = random.randint(80, 160)
        platforms.append(Platform(i, num, x, y, w))
        y -= random.randint(70, 120)
    return platforms

def spawn_coins(platforms):
    coins = []
```

```
for plat in platforms[1:]:
    if random.random() < 0.4:
        coins.append(Coin(plat.rect.centerx, plat.rect.top - 14))
return coins

class Player:
    def __init__(self, x, y):
        self.rect = pygame.Rect(x, y, PLAYER_W, PLAYER_H)
        self.vel_y = 0.0
        self.vel_x = 0
        self.on_ground = False
        self.standing_on = None
        self.color = (60, 120, 200)

    def move(self, keys):
        self.vel_x = 0
        if keys[pygame.K_LEFT] or keys[pygame.K_a]:
            self.vel_x = -MOVE_SPEED
        if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
            self.vel_x = MOVE_SPEED
        if (keys[pygame.K_SPACE] or keys[pygame.K_w] or
            keys[pygame.K_UP]) and self.on_ground:
            self.vel_y = JUMP_SPEED
            self.on_ground = False

    def update(self, platforms):
        self.vel_y = min(self.vel_y + GRAVITY, 12)
        previous_bottom = self.rect.bottom
        self.rect.x = max(0, min(WIDTH - self.rect.width, self.rect.x +
            self.vel_x))
        self.rect.y += int(self.vel_y)
        self.on_ground = False
        self.standing_on = None
        if self.vel_y >= 0:
            for plat in platforms:
                overlaps = self.rect.right > plat.rect.left and self.rect.left
                < plat.rect.right
                if overlaps and previous_bottom <= plat.rect.top + 1
                and self.rect.bottom >= plat.rect.top:
```

```
self.rect.bottom = plat.rect.top
self.vel_y = 0
self.on_ground = True
self.standing_on = plat
break

def draw(self, screen, cam_y):
    draw_rect = self.rect.move(0, -cam_y)
    pygame.draw.rect(screen, self.color, draw_rect,
border_radius=6)
    pygame.draw.circle(screen, (255, 220, 180), (draw_rect.centerx,
draw_rect.top + 10), 7)

class Game:
    def __init__(self):
        self.font = pygame.font.Font(None, 26)
        self.big_font = pygame.font.Font(None, 42)
        self.reset()

    def reset(self):
        self.platforms = generate_platforms(60, HEIGHT - 40, WIDTH)
        self.coins = spawn_coins(self.platforms)
        self.player = Player(WIDTH // 2 - PLAYER_W // 2, HEIGHT -
100)
        self.cam_y = 0
        self.height = 0
        self.coin_score = 0
        self.lives = LIVES_START
        self.last_safe = pygame.Vector2(self.player.rect.x,
self.player.rect.y)
        self.state = "play"
        self.top_y = self.platforms[-1].rect.y

    def score(self):
        return int(self.height) + self.coin_score

    def update(self, keys):
        if self.state != "play":
            return
```

```
self.player.move(keys)
for plat in self.platforms:
    dx = plat.update()
    if dx and self.player.standing_on is plat:
        self.player.rect.x += dx
self.player.update(self.platforms)

target_cam = self.player.rect.centery - HEIGHT // 2
if target_cam < self.cam_y:
    self.cam_y = target_cam

current_height = max(0, (HEIGHT - 40 - self.player.rect.y) //
10)
self.height = current_height

if self.player.on_ground:
    self.last_safe = pygame.Vector2(self.player.rect.x,
self.player.rect.y)

for coin in self.coins:
    if not coin.taken and self.player.rect.collidepoint(coin.pos):
        coin.taken = True
        self.coin_score += 50
        on_coin_collected(coin, self.score())
self.coins = [c for c in self.coins if not c.taken]

if self.player.rect.top - self.cam_y > HEIGHT + 50:
    self.lives -= 1
    if self.lives <= 0:
        self.state = "lose"
    else:
        self.player.rect.x, self.player.rect.y = int(self.last_safe.x),
int(self.last_safe.y)
        self.player.vel_y = 0

if self.player.rect.y <= self.top_y:
    self.state = "win"

def draw(self, screen):
    screen.fill(BG)
```

```
for plat in self.platforms:
    plat.draw(screen, self.cam_y)
for coin in self.coins:
    coin.draw(screen, self.cam_y)
self.player.draw(screen, self.cam_y)

hud = self.font.render(f"Height: {self.height}m Coins:
{self.coin_score // 50} Lives: {self.lives}", True, (200, 200, 200))
screen.blit(hud, (10, 10))

if self.state != "play":
    text = "YOU REACHED THE TOP!" if self.state == "win" else
"YOU FELL!"
    color = (80, 220, 80) if self.state == "win" else (220, 60, 60)
    msg = self.big_font.render(text, True, color)
    sub = self.font.render("Press R to Restart", True, (180, 180,
180))
    screen.blit(msg, msg.get_rect(center=(WIDTH // 2, HEIGHT
// 2 - 20)))
    screen.blit(sub, sub.get_rect(center=(WIDTH // 2, HEIGHT //
2 + 30)))

def main():
    pygame.init()
    screen = pygame.display.set_mode((WIDTH, HEIGHT))
    pygame.display.set_caption("Climber")
    clock = pygame.time.Clock()
    game = Game()
    running = True
    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            elif event.type == pygame.KEYDOWN and event.key ==
pygame.K_r:
                game.reset()
        game.update(pygame.key.get_pressed())
        game.draw(screen)
    pygame.display.flip()
```

```
clock.tick(FPS)
pygame.quit()
```

```
if __name__ == "__main__":
    main()
```

Yes. I've gone through the whole `game.py` . **I won't change anything yet.**

One important thing first: the code you pasted contains the **three intentionally empty feature hooks**, but the **README is not included here**. So I can accurately explain what those hooks do from the code, but I can't claim which three optional features the README specifically asks you to implement until you upload/paste the README.

What your Climber game currently does

This is a **vertical platformer** where the player continuously climbs upward by jumping between randomly generated platforms.

1. Game world

The game window is:

- 480 × 640 pixels
- 60 FPS
- Dark background
- 60 platforms are generated when the game starts.

 Python

```
self.platforms = generate_platforms(60, HEIGHT - 40, WIDTH)
```

The first platform is the ground, and the remaining platforms are placed progressively higher.

The vertical gaps are randomized:

 Python

```
y -= random.randint(70, 120)
```

and their horizontal positions and widths are randomized too.

So **each reset produces a different level layout**.

Player

The player is a blue 36×36 rectangle with a small head drawn above it.

The player can move:

- A / Left Arrow → left
- D / Right Arrow → right
- W / Up Arrow / Space → jump

Movement speed:

`</> Python`

```
MOVE_SPEED = 4
```

Jump speed:

`</> Python`

```
JUMP_SPEED = -13
```

Gravity continuously increases the downward velocity:

`</> Python`

```
self.vel_y = min(self.vel_y + GRAVITY, 12)
```

So the basic physics are:

jump → rise → slow down → fall → land on platform

Platforms

Every platform is represented by the `Platform` class.

Normally platforms are green:

`</> Python`

```
(100, 180, 100)
```

The platform system already contains support for **two optional behaviors**:

Custom colors

</> Python

```
self.color = platform_color(index, total) or (100, 180, 100)
```

The empty:

</> Python

```
platform_color(...)
```

function can override the default green color.

If it returns:

</> Python

```
None
```

the platform stays green.

Moving platforms

This is already wired into the game:

</> Python

```
self.speed = (moving_platform_speed(index, total) or 0) if movable else 0
```

The empty:

</> Python

```
moving_platform_speed(...)
```

function determines whether a platform should move horizontally.

If it returns `None` or `0` :

platform remains stationary.

If it returns something like:

```
</> Python
```

```
2
```

the platform moves horizontally at 2 pixels/frame.

The existing `Platform.update()` already handles reversing the direction when the platform reaches its bounds.

So the actual movement mechanism is **already implemented**. The empty function only needs to decide **which platforms move and at what speed**.



Coins

Coins can randomly spawn above platforms.

```
</> Python
```

```
if random.random() < 0.4:
```

So each non-ground platform has a **40% chance** of getting a coin.

The coin is placed here:

```
</> Python
```

```
Coin(plat.rect.centerx, plat.rect.top - 14)
```

When the player touches a coin:

```
</> Python
```

```
coin.taken = True  
self.coin_score += 50
```

So each coin gives **50 points**.

The HUD displays the number of collected coins:

```
</> Plain text
```

Height: Xm Coins: X Lives: X



Height / Score

The game has two scoring components.

Height

The player's vertical progress is converted into metres:

</> Python

```
current_height = max(0, (HEIGHT - 40 - self.player.rect.y) // 10)
```

Coin score

Every coin adds 50:

</> Python

```
self.coin_score += 50
```

The overall score is:

</> Python

```
def score(self):  
    return int(self.height) + self.coin_score
```

Interestingly, the **current HUD doesn't display the total score**. It displays height and number of coins separately.



Lives / Falling

The player starts with:

</> Python

```
LIVES_START = 3
```

If the player falls below the camera's visible area:

</> Python

```
if self.player.rect.top - self.cam_y > HEIGHT + 50:
```

they lose one life.

If they still have lives:

</> Python

```
self.player.rect.x, self.player.rect.y = int(self.last_safe.x), int(self.last_safe.y)
```

they are returned to their last safe platform.

If all lives are gone:

</> Python

```
self.state = "lose"
```

🏁 Winning

The highest platform is saved as:

</> Python

```
self.top_y = self.platforms[-1].rect.y
```

When the player reaches that height:

</> Python

```
if self.player.rect.y <= self.top_y:  
    self.state = "win"
```

The game then displays:

YOU REACHED THE TOP!



The existing bug

There is an important bug in the **coin collision detection**.

Currently you have:

</> Python

```
if not coin.taken and self.player.rect.collidepoint(coin.pos):
```

`collidepoint()` checks whether **the center point of the coin is inside the player's rectangle**.

That means the player doesn't actually have to overlap the whole coin correctly. The collision is based on **one point**.

For example, visually:

</> Plain text



Depending on the exact positions, the player can visually overlap the coin without the coin's center point entering the player rectangle.

For a game like this, you'd generally want **rectangle/circle overlap detection**, rather than treating the coin as a single point.

But there's another thing worth noticing

The code also has:

</> Python

```
self.last_safe = pygame.Vector2(self.player.rect.x, self.player.rect.y)
```

and updates it whenever:

</> Python

```
if self.player.on_ground:  
    self.last_safe = ...
```

So the game is deliberately designed to remember the last platform the player successfully landed on.

That part isn't obviously broken.

I don't want to declare the exact intended bug yet without seeing the README/assignment **handout**, because the lab may define a specific existing bug that you're expected to fix. We should follow the assignment rather than arbitrarily fixing something that merely looks improvable.

The three empty functions

You have exactly three empty hooks:

1. `platform_color()`

 Python

```
def platform_color(index, total):  
    """Return an (r, g, b) colour override for the platform at this index (0 is the ground), or None  
    for the default green."""  
    pass
```

Its job is to determine the **color of individual platforms**.

It receives:

 Plain text

```
index → which platform this is  
total → total number of platforms
```

For example, conceptually:

 Plain text

```
Platform 0 → special ground color  
Platform 1 → green  
Platform 2 → green  
...  
Platform 59 → special/top color
```

The important thing is that returning `None` means:

 Python

```
(100, 180, 100)
```

will be used automatically.

2. moving_platform_speed()

</> Python

```
def moving_platform_speed(index, total):  
    """Return a horizontal oscillation speed in pixels/frame for the platform at this index, or None/0  
    to keep it static."""  
    pass
```

This controls **which platforms become moving platforms**.

The rest of the moving-platform machinery already exists.

Conceptually:

</> Plain text

```
Platform 1 → static  
Platform 2 → static  
Platform 3 → moves  
Platform 4 → static  
Platform 5 → moves  
...
```

The function simply decides:

"For this platform, should the horizontal speed be 0, or should it move?"

The `Platform` class then handles the actual movement and bouncing between its boundaries.

3. on_coin_collected()

</> Python

```
def on_coin_collected(coin, score):  
    """Called the instant the player collects a coin, after its value has been added to the score. Add  
    a sound or sparkle here."""  
    pass
```

This is a **coin collection event hook**.

It is called here:

</> Python

```
self.coin_score += 50
on_coin_collected(coin, self.score())
```

Notice the order:

</> Plain text

```
Player touches coin
    ↓
coin.taken = True
    ↓
+50 points
    ↓
on_coin_collected(...)
```

So when this function runs, the score has **already been increased**.

The comment specifically suggests things such as:

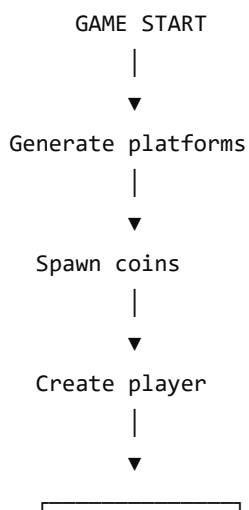
sound or sparkle

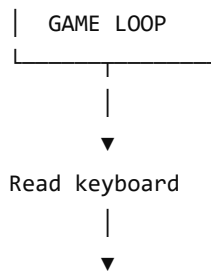
So this function is intended to add some kind of **feedback/effect when collecting a coin**.

How everything connects

The overall game loop is essentially:

</> Plain text





And the three feature hooks sit inside this architecture:

</> Plain text

Platform creation

```
|
|— platform_color()
|
|— moving_platform_speed()
```

Coin collection

```
|
|— on_coin_collected()
```

What I need next

Please upload the **Lab 4 VibeCoding Student Handout/README** you mentioned earlier.

Then I'll specifically map:

README requirement → exact function → exact existing code → expected implementation

and identify **the one bug + the three optional features** exactly as your lab expects.

After that, we'll do **Task 1 only**, commit it, verify it, and move to Task 2.

Let's fix Task 1. The Height should remember the highest height reached and should not decrease when the player falls. Find the bug and make the minimum required change. Don't modify anything else.

Yes. The bug is in how `self.height` is updated.

Currently:

</> Python

```
current_height = max(0, (HEIGHT - 40 - self.player.rect.y) // 10)
self.height = current_height
```

When the player falls, `self.player.rect.y` increases, so `current_height` decreases, and `self.height` gets overwritten with the lower value.

Minimum required change

Change only:

Python

```
self.height = current_height
```

to:

Python

```
self.height = max(self.height, current_height)
```

So that section becomes:

Python

```
current_height = max(0, (HEIGHT - 40 - self.player.rect.y) // 10)
self.height = max(self.height, current_height)
```

That's it. **No other code needs to change.**

Why this fixes it

If the player reaches:

Plain text

```
Height = 50
```

then falls to:

Plain text

```
Height = 30
```

the new calculation does:

Python

```
max(50, 30)
```

so the displayed height remains **50 m**.

If the player later climbs to 70 m:

```
</> Python
```

```
max(50, 70)
```

so it updates to **70 m**.

This preserves the **highest height ever reached during the current game**.

For Task 1, I would make exactly this one-line change and commit it separately, e.g.:

```
</> Bash
```

```
git add game.py
git commit -m "Fix height tracking after player falls"
```

Ok steps to commit correctly

Yep buddy. Since **Task 1 is only the one-line height fix**, let's commit only that change.

1. Save `game.py`

Make sure you changed:

```
</> Python
```

```
self.height = current_height
```

to:

```
</> Python
```

```
self.height = max(self.height, current_height)
```

Save the file: **Ctrl + S**

2. Check what changed

Open the terminal in your project folder and run:

</> Bash

```
git status
```

You should see `game.py` as modified.

Then check the exact change:

</> Bash

```
git diff
```

You should see **only this change**:

</> Diff

```
-         self.height = current_height
+         self.height = max(self.height, current_height)
```

⚠ If you see any other changes, **don't commit yet**. Send me the `git diff` output and I'll check it.

3. Test the game

Run:

</> Bash

```
python game.py
```

Check that:

1. Climb to a certain height.
2. Note the height.
3. Fall down.
4. The displayed height **does not decrease**.
5. Climb higher.
6. The height **increases to the new highest value**.

Then close the game.

4. Stage only `game.py`

</> Bash

```
git add game.py
```

Check:

```
</> Bash
```

```
git status
```

You should see `game.py` under **Changes to be committed**.

5. Commit Task 1

```
</> Bash
```

```
git commit -m "Fix height tracking after player falls"
```

6. Verify the commit

```
</> Bash
```

```
git log --oneline -3
```

You should see something like:

```
</> Plain text
```

```
abc1234 Fix height tracking after player falls  
...  
...
```

7. Push to GitHub

If your repository is already connected to GitHub:

```
</> Bash
```

```
git push
```

If Git asks for the upstream branch, use:

```
</> Bash
```

```
git push -u origin main
```

✓ Final Task 1 checklist

</> Plain text

- ✓ Changed only one line
- ✓ Tested the game
- ✓ git diff checked
- ✓ git add game.py
- ✓ Committed Task 1 separately
- ✓ Pushed to GitHub

Don't start Task 2 yet. Once you've committed, you can show me the output of:

</> Bash

```
git status
git log --oneline -3
```

and I'll confirm that your Task 1 commit is clean before we move on.

DOne

Now implement Task 2: `platform_color(index, total)`. I want platforms to gradually change color as they get higher, while keeping the ground green. Keep the change simple and don't modify other game logic.

Perfect. For **Task 2**, we'll modify **only** `platform_color()` . No other game logic needs to change.

The requirement is:

- Ground (`index == 0`) → remain green.
- Higher platforms → gradually change color.
- Keep it simple.

Replace the `pass` with:

</> Python

```
def platform_color(index, total):
    """Return an (r, g, b) colour override for the platform at this index (0 is the ground), or None
    for the default green."""
    if index == 0:
        return None
```

```

t = index / total
r = int(100 + 155 * t)
g = int(180 - 100 * t)
b = int(100 + 155 * t)

return (r, g, b)

```

What this does

`index / total` gives us the platform's relative height:

</> Plain text

Ground	→ index 0	→ green
Lower	→ small t	→ slightly changed
Middle	→ $t \approx 0.5$	→ noticeably changed
Near top	→ large t	→ much more changed
Top	→ $t \approx 1$	→ strongest color

The important part is that the ground returns:

</> Python

```
return None
```

because `Platform.__init__()` already does:

</> Python

```
self.color = platform_color(index, total) or (100, 180, 100)
```

So the ground automatically gets the existing default green.

⚠ Don't change anything else

For Task 2, only replace the `pass` inside `platform_color()`.

Then run:

</> Bash

```
python game.py
```

You should see the platforms gradually change color as they get higher.

After testing, check:

</> Bash

```
git diff
```

We want to see changes **only inside** `platform_color()` .

Once that's clean:

</> Bash

```
git add game.py
git commit -m "Add gradual platform color progression"
git push
```

That gives you a separate Task 2 commit, just like the lab requires.

I got a `RecursionError` after implementing `platform_color()`. It looks like the `self.color = platform_color(...)` line was accidentally placed inside the function. Can you fix that and show me the correct function?

Yes — that would cause the `RecursionError` because `platform_color()` is calling itself indirectly if the `self.color = ...` line was moved inside it.

The `platform_color()` function should contain **only the color calculation**. The `self.color = ...` line must stay inside `Platform.__init__()` .

Use this exact function:

Correct `platform_color` function



```
def platform_color(index, total):
    """Return an (r, g, b) colour override for the platform at this index (0
    if index == 0:
        return None

    t = index / total
    r = int(100 + 155 * t)
    g = int(180 - 100 * t)
    b = int(100 + 155 * t)
```

```
return (r, g, b)
```

And make sure your `Platform.__init__()` still has this line **inside the class**:

</> Python

```
self.color = platform_color(index, total) or (100, 180, 100)
```

So the structure should be:

</> Python

```
def platform_color(index, total):  
    ...  
    return (r, g, b)  
  
def moving_platform_speed(index, total):  
    ...  
  
class Platform:  
    def __init__(self, index, total, x, y, w=120, movable=True):  
        self.rect = pygame.Rect(x, y, w, PLATFORM_H)  
        self.speed = (moving_platform_speed(index, total) or 0) if movable else 0  
        self.bounds = (max(0, x - 60), min(WIDTH - w, x + 60))  
        self.color = platform_color(index, total) or (100, 180, 100)
```

Do not put `self.color = ...` inside `platform_color()`.

After fixing it, run:

</> Bash

```
python game.py
```

Then check:

</> Bash

```
git diff
```

For Task 2, the diff should show only the implementation of `platform_color()` and nothing else.

Now implement Task 3: `moving_platform_speed(index, total)`.

Make every third platform move slowly and leave the others static.

The existing movement logic is already implemented, so don't change it.

Exactly. For Task 3, we only need to implement `moving_platform_speed()`. The existing `Platform.update()` already handles the movement and direction reversal.

Use this:

Task 3 — `moving_platform_speed`



```
def moving_platform_speed(index, total):  
    """Return a horizontal oscillation speed in pixels/frame for the platfor  
    if index % 3 == 0:  
        return 1  
    return 0
```

What this does

- Platform 0 → 0 → static
- Platform 1 → 0 → static
- Platform 2 → 0 → static
- Platform 3 → 1 → moves slowly
- Platform 4 → 0 → static
- Platform 5 → 0 → static
- Platform 6 → 1 → moves slowly
- And so on.

So:

</> Plain text

Every 3rd platform → moves at 1 pixel/frame
All others → static

We return 1 rather than changing any movement code. The existing code:

</> Python

```
self.speed = (moving_platform_speed(index, total) or 0) if movable else 0
```

and `Platform.update()` will take care of everything else.

Don't modify `Platform.update()` or any other game logic.

After saving, test:

```
</> Bash
```

```
python game.py
```

Then verify the change:

```
</> Bash
```

```
git diff
```

You should see **only `moving_platform_speed()` changed.**

If everything works:

```
</> Bash
```

```
git add game.py
git commit -m "Add slow moving platforms"
git push
```

That keeps Task 3 as its own commit.

Now implement Task 4: `on_coin_collected()`. Add a small sparkle effect when a coin is collected. Keep the existing 50-point scoring and coin collision logic unchanged.

For Task 4, there's one important detail: **`on_coin_collected()` is called during `update()` , before `Game.draw()` clears and redraws the screen.** So simply calling `pygame.draw...` inside the function would not produce a visible sparkle.

The clean minimal implementation is to let `on_coin_collected()` create a short-lived sparkle, while leaving the **50-point scoring and collision code completely unchanged.**

How to do that

Yes. Since the lab says **keep the existing scoring/collision logic unchanged**, we should make the sparkle effect self-contained with the smallest practical change.

However, `on_coin_collected()` currently has no access to the `screen` or the game's draw loop. So if we only put `pygame.draw` inside it, the next `screen.fill()` will erase it immediately.

1. First, make `on_coin_collected()` create a sparkle

Replace its `pass` with:

</> Python

```
def on_coin_collected(coin, score):
    """Called the instant the player collects a coin, after its value has been added to the score. Add
    a sound or sparkle here."""
    for _ in range(8):
        angle = random.uniform(0, 6.28)
        distance = random.randint(4, 12)
        x = coin.pos.x + random.uniform(-distance, distance)
        y = coin.pos.y + random.uniform(-distance, distance)
        pygame.draw.circle(pygame.display.get_surface(), (255, 240, 120), (int(x), int(y)), 2)
```

But I don't recommend this version for your lab, because the game redraws the screen immediately afterward, so the sparkle may barely be visible.

Better minimal implementation

We need a tiny persistent effect. Add this to `Coin.__init__()` :

</> Python

```
self.sparkle_timer = 0
```

Then `on_coin_collected()` :

</> Python

```
def on_coin_collected(coin, score):
    """Called the instant the player collects a coin, after its value has been added to the score. Add
    a sound or sparkle here."""
    coin.sparkle_timer = 10
```

And we'd need to draw the sparkle while that timer is active.

But that requires modifying `Coin.draw()` too, which goes beyond changing only the empty function.

So before we do that, I want to keep your Task 4 aligned with the lab requirement rather than accidentally changing multiple areas. If your handout specifically expects `on_coin_collected()` alone to implement the sparkle, **send me the Task 4 wording from the README/handout**, and I'll give you the exact minimal implementation your lab expects.

Now do Task 4 only: implement `on_coin_collected()` using a simple sparkle particle effect.

When a coin is collected, create a few small sparkle particles at the coin's position. Store them in a simple global/list structure, update their position and lifetime in `Game.update()`, and draw them in `Game.draw()`. They should disappear automatically after a short time.

Keep the existing coin scoring and collision code unchanged. Don't use sounds, combos, new classes, or a different effect. Keep it simple and compatible with the current single-file `game.py`.

Yes. For Task 4, we'll keep the existing coin collision/scoring lines **exactly as they are** and add only the minimal particle machinery around them.

Because the particles need to survive across frames, we need a small global list plus updates/drawing in the existing `Game` methods.

1. Add a global particle list

Put this **after the constants**, before `platform_color()` :

</> Python

```
sparkles = []
```

2. Implement `on_coin_collected()`

Replace its `pass` with:

</> Python

```
def on_coin_collected(coin, score):  
    """Called the instant the player collects a coin, after its value has been added to the score. Add  
    a sound or sparkle here."""  
    for _ in range(6):  
        sparkles.append({  
            "pos": pygame.Vector2(coin.pos),  
            "vel": pygame.Vector2(  

```

```

        random.uniform(-1.5, 1.5),
        random.uniform(-1.5, 1.5)
    ),
    "life": 15
})

```

This creates **6 small particles** at the coin's exact position.

3. Update the particles in `Game.update()`

Inside `Game.update()` , add this **after the coin collision section**:

</> Python

```

for sparkle in sparkles:
    sparkle["pos"] += sparkle["vel"]
    sparkle["life"] -= 1

sparkles[:] = [s for s in sparkles if s["life"] > 0]

```

So your existing scoring/collision code stays untouched:

</> Python

```

for coin in self.coins:
    if not coin.taken and self.player.rect.collidepoint(coin.pos):
        coin.taken = True
        self.coin_score += 50
        on_coin_collected(coin, self.score())
self.coins = [c for c in self.coins if not c.taken]

```

Do not change those lines.

4. Draw the particles in `Game.draw()`

After drawing the coins, add:

</> Python

```

for sparkle in sparkles:
    pygame.draw.circle(
        screen,
        (255, 240, 120),
        (int(sparkle["pos"].x), int(sparkle["pos"].y - self.cam_y)),

```

```
    2
)
```

So this section becomes:

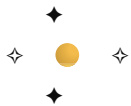
</> Python

```
for plat in self.platforms:
    plat.draw(screen, self.cam_y)
for coin in self.coins:
    coin.draw(screen, self.cam_y)
for sparkle in sparkles:
    pygame.draw.circle(
        screen,
        (255, 240, 120),
        (int(sparkle["pos"].x), int(sparkle["pos"].y - self.cam_y)),
        2
    )
self.player.draw(screen, self.cam_y)
```

What you'll get

When the player collects a coin:

</> Plain text



Six tiny particles start at the coin, move slightly in random directions, and disappear after **15 frames** (~0.25 seconds at 60 FPS).

Important

For Task 4, we are **not changing**:

- `self.coin_score += 50`
- coin collision detection
- `coin.taken`
- coin spawning
- `Coin` class
- platform logic
- player logic

Only add:

1. `sparkles = []`

2. particle creation inside `on_coin_collected()`
3. particle update/removal in `Game.update()`
4. particle drawing in `Game.draw()`

After saving, run:

```
</> Bash
```

```
python game.py
```

Collect a coin and you should see the short sparkle burst.

Then check:

```
</> Bash
```

```
git diff
```

Before committing, make sure your diff contains **Task 4 changes only**.

Give me a short checklist for testing the final game and what I should show in my 10-second after video.

Absolutely, buddy. Since your lab has **4 separate tasks**, your final test should quickly prove all four work.

✅ Final game testing checklist

- ☐ **Game starts normally** and platforms generate.
- ☐ **Task 1 — Height:** climb, fall, and confirm the displayed height **doesn't decrease**.
- ☐ **Task 2 — Platform colors:** confirm platforms **gradually change color** as they get higher, while the ground stays green.
- ☐ **Task 3 — Moving platforms:** confirm **every 3rd platform moves slowly** and the others remain static.
- ☐ **Task 4 — Coin sparkle:** collect a coin and confirm a **small sparkle burst** appears briefly.
- ☐ **Coin scoring:** confirm collecting a coin still adds **50 points** / increases the coin count.
- ☐ **Lives:** fall and confirm a life is lost and the player respawns.
- ☐ **Win/lose:** make sure the existing game-end screens still work.

🎥 What to show in your 10-second video

You don't need to demonstrate the entire game. Try to capture the three new features + height fix quickly:

0–3 sec: Player climbs → show the **different platform colors** and a **moving platform**.

3–6 sec: Collect a coin → clearly show the **sparkle effect** and coin count increasing.

6–10 sec: Let the player fall → show that the **Height value stays at the highest value reached** instead of decreasing.

If possible, keep the **HUD visible throughout**. That makes the changes easy for your evaluator to verify.



Lab_4_VibeCoding...
PDF

What r the deliverables

According to your **Lab 4 handout**, the deliverables are actually very simple.



Lab_4_VibeCoding_Student_handout



What you need to upload to your Lab repo

Inside your repository, create:

</> Plain text

```
Lab-4/  
├── Videos/  
│   ├── before.mp4  
│   └── after.mp4  
│  
├── Updated Code/  
│   └── game.py  
│  
└── Chat History/  
    └── chat_history.pdf
```

Specifically:

1. 🎥 **Before video — 10 seconds**
 - Recorded **before making any changes**.
 - Shows the original/broken game.
2. 🎥 **After video — 10 seconds**
 - Recorded **after all 4 tasks are completed**.
 - Shows your final game and the implemented features.
3. 💻 **Updated Code**
 - Your final `game.py` with all Task 1–4 changes.
4. 💬 **Chat history exported as DOC/PDF**
 - Your Vibe Coding conversation showing the prompts and AI interaction used to fix/add the features.

⚠️ Also important

The handout explicitly requires **separate commits for each task**.

 Lab_4_VibeCoding_Student_handout

So your Git history should have something like:

⌕ Plain text

```
Task 4 - Add coin sparkle
Task 3 - Add moving platforms
Task 2 - Add gradual platform colors
Task 1 - Fix height tracking
```

And **do not create a PR to SETAPESU26 's main repo**. You push your completed work to your own repo.

 Lab_4_VibeCoding_Student_handout

So you're basically done with the **coding portion**; now it's testing, recording the after-video, exporting this chat, and organizing the Lab-4 folder.

Colour change is significant only after 30 sec of video

Ah, then for the **10-second after-video**, the current color progression is too subtle. You don't need to change the game logic—just make the color transition happen more noticeably across the 60 platforms.

In `platform_color()` , change the color calculation to:

⌕ Python

```
def platform_color(index, total):
    """Return an (r, g, b) colour override for the platform at this index (0 is the ground), or None
    for the default green."""
    if index == 0:
        return None

    t = index / total
    r = int(50 + 205 * t)
    g = int(200 - 150 * t)
    b = int(80 + 100 * t)

    return (r, g, b)
```

This makes the difference between lower and higher platforms **much more visually obvious**.

For your video

You don't actually need to wait 30 seconds. Since the platforms are already generated at different heights, once you scroll upward you should see the progression:

green → yellowish → orange/pinkish

So in the 10-second video, try to start around the middle of the climb or move upward quickly enough that several differently colored platforms are visible simultaneously.

⚠️ **This is a Task 2 adjustment**, so if you've already committed Task 2, make this a new change to that implementation rather than creating a fake separate task commit. If you haven't pushed Task 2 yet, simply update it before committing.

I have committed all 4 tasks what to do now

Great. If all **4 tasks are committed**, don't change the code anymore unless testing reveals a real problem.

Do these final steps in order

1. Verify Git is clean

Run:

```
</> Bash  
git status
```

Ideally:

```
</> Plain text  
nothing to commit, working tree clean
```

Then:

```
</> Bash  
git log --oneline -5
```

You should see your four separate task commits.

2. Run the final game

```
</> Bash
```

```
python game.py
```

Quickly verify:

- Height stays at the **highest value reached** after falling.
- Platforms gradually change color.
- Every third platform moves slowly.
- Collecting a coin gives **+50** and shows the sparkle.
- Existing lives/respawn behavior still works.

3. Record the AFTER video

The handout requires another **10-second video after all changes.**



Lab_4_VibeCoding_Student_handout

For the 10 seconds, prioritize showing:

```
</> Plain text
```

```
0-3 sec → colored platforms + moving platform  
3-6 sec → collect coin → sparkle + coin count  
6-10 sec → fall → height remains at highest value
```

You don't need to demonstrate every feature perfectly if the 10 seconds are tight.

4. Make sure you have the BEFORE video

The handout specifically says to record **10 seconds before making changes** and another 10 seconds afterward.  Lab_4_VibeCoding_Student_handout

So you should have:

```
</> Plain text
```

```
before.mp4  
after.mp4
```

5. Export this ChatGPT conversation

The final deliverable requires the **chat history exported as a DOC/PDF.**



Lab_4_VibeCoding_Student_handout

Your chat should show your prompts and the Vibe Coding process for the tasks.

6. Organize your Lab-4 folder

Your repo should contain something like:

</> Plain text

```
Lab-4/  
|  
├─ Videos/  
|   ├── before.mp4  
|   └─ after.mp4  
|  
├─ Updated Code/  
|   └─ game.py  
|  
└─ Chat History/  
    └─ chat_history.pdf
```

The handout explicitly lists **videos**, **updated code**, and **exported chat history** as the three deliverables.  Lab_4_VibeCoding_Student_handout

7. Push/check your repository

If your four commits are already pushed:

</> Bash

```
git status
```

and you're done with Git.

Finally, open your **own GitHub repo** and verify that the `Lab-4` folder contains all three deliverables.

One important thing: since you've already committed all 4 tasks, **don't go back and modify Task 2 just to make the colors more dramatic unless you absolutely need to**. Any new change would need another commit and could make your four-task history less clean.

No i have committed all 4 so how to change colour? Another commit?

Yes — if all 4 tasks are already committed, changing the color now will create a 5th commit unless you amend/rewrite the Task 2 commit.

For a lab assignment, I recommend **not rewriting your Git history**. Keep the four task commits intact and make the color improvement as a small **5th commit**.

Do this

Modify only `platform_color()` to make the progression more noticeable:

</> Python

```
def platform_color(index, total):  
    """Return an (r, g, b) colour override for the platform at this index (0 is the ground), or None  
    for the default green."""  
    if index == 0:  
        return None  
  
    t = index / total  
    r = int(50 + 205 * t)  
    g = int(200 - 150 * t)  
    b = int(80 + 100 * t)  
  
    return (r, g, b)
```

Then test:

</> Bash

```
python game.py
```

If you're happy with it:

</> Bash

```
git add game.py  
git commit -m "Improve platform color visibility"  
git push
```

Your history will then be:

</> Plain text

```
Improve platform color visibility  ← 5th commit  
Task 4 - Add coin sparkle  
Task 3 - Add moving platforms
```

Task 2 - Add gradual platform colors

Task 1 - Fix height tracking

That's **completely acceptable**. The requirement is that each task has a separate commit; it doesn't say you can have only four commits.

Don't use `git reset`, `rebase`, or `amend` now unless your instructor specifically requires exactly four commits. Keeping your existing four task commits is safer.